



PDF Download
3747204.3747210.pdf
16 March 2026
Total Citations: 0
Total Downloads: 950



Published: 18 August 2025

Citation in BibTeX format

ICNS3 2025: 2025 International
Conference on ns-3
August 19 - 21, 2025
Osaka, Japan

 Latest updates: <https://dl.acm.org/doi/10.1145/3747204.3747210>

RESEARCH-ARTICLE

P4sim: Simulating Programmable Switches in ns-3

MINGYU MA, TUD Dresden University of Technology, Dresden, Sachsen, Germany

GIANG T. NGUYEN, TUD Dresden University of Technology, Dresden, Sachsen, Germany

Open Access Support provided by:

TUD Dresden University of Technology

P4sim: Simulating Programmable Switches in ns-3

Mingyu Ma

Dresden University of Technology
Dresden, Saxony, Germany
mingyu.ma@tu-dresden.de

Giang T. Nguyen

Dresden University of Technology
Dresden, Saxony, Germany
giang.nguyen@tu-dresden.de

Abstract

Programming Protocol-Independent Packet Processors (P4) has enabled network researchers to program packet processing pipelines to prototype and implement novel network functions in hardware and software testbeds. P4 support in simulators, like network simulator version 3 (ns-3) can benefit significantly from ns-3's feature-rich ecosystem. However, existing P4 simulators have poor scalability, inaccurate timing, and limited architectural support. This paper introduces P4sim, a high-performance simulator developed on top of ns-3. P4sim enhances queue modeling and packet scheduling, ensuring efficient packet processing and precise simulation time representation. The evaluation results confirm that P4sim effectively manages packet processing, supporting data rates up to 10 Gbps. Furthermore, P4sim supports various architectures, like V1Model, Portable Switch Architecture (PSA), and Portable NIC Architecture (PNA). It enables seamless transition between Behavioral Model version 2 (bmv2) and P4sim, allowing researchers to shift from rapid prototyping to large-scale network simulations with the same P4 code.

CCS Concepts

• Networks → Network simulations; Programmable networks.

Keywords

simulator, ns-3, P4, bmv2, V1Model, programmable data plane

ACM Reference Format:

Mingyu Ma and Giang T. Nguyen. 2025. P4sim: Simulating Programmable Switches in ns-3. In *2025 International Conference on ns-3 (ICNS3 2025)*, August 19–21, 2025, Osaka, Japan. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3747204.3747210>

1 Introduction

The dominance of proprietary networking solutions limits interoperability and innovation. Traditional networking hardware, such as switches and routers, relies on fixed-function Application-Specific Integrated Circuit (ASIC) tailored to specific protocols like the Ethernet protocol. This rigidity made it difficult to adapt to new protocols or custom requirements without costly hardware upgrades. As networks grow more diverse, supporting an ever-expanding protocols has become a challenge. Programmable data planes, powered by the P4 language, allow network behavior to be defined and modified via software, enabling rapid innovation and customization. P4

allows for writing a single P4 program and adapting it to various hardware targets, such as NetFPGA, or Network Processing Unit (NPU), by pairing it with an appropriate architecture and compiler back-end. However, building a hardware testbed, especially in a large scale require substantial investment, not only in money, but also in expertise and time. Even more flexible with the reference software switch, bmv2, enables rapid prototyping. bmv2 simulates the behavior of a programmable data plane, allowing developers to test P4 programs without requiring physical hardware. However, inherits software-related limitations, bmv2 demands high computational overhead and poor scalability [1, 2]. This limitation hinders the study of programmable data plane behavior on complex and large-scale network topologies. As an alternative, supporting P4 functionalities in an established and feature-rich network simulator like ns-3 can open a new dimension of possibilities.

The most notable works on simulating P4 switch in ns-3 include ns3-bmv2 [3], NetDevices [4], and NS4 [5, 6]. These works have made substantial advances to support P4 software targets in ns-3. However, they demonstrate several drawbacks. ns3-bmv2 lacks routing and switch-level processing—the core P4 functionalities. NetDevices introduce a complex integration, hindering tracing and debugging. NS4 does not support event scheduling and modern P4 architectures. These limitations hinder its effectiveness in simulating programmable networks. Supporting P4 in ns-3 poses several challenges. The lack of switching and duplex link modules, and the lack of queueing models for switches, restrict their ability to support essential mechanisms such as scheduling, buffering, and traffic management. Moreover, the absence of virtual time representation hinders the precise modeling of packet transmission, delays, and scheduling between different network components [7]. This prevents ns-3 switches from accurately performing essential functions such as monitoring and congestion control.

We introduce P4sim^{1 2}, a high-performance P4-driven simulation framework for ns-3. P4sim improves switch modeling, time representation, and extensibility. Our key contributions are as follows. *i) High-performance event scheduling*: by efficient queue modeling within switches, optimizing packet processing for realistic high-speed links. *ii) Extended P4 support*: to various modern P4 architectures (V1Model, PSA, and PNA), allowing simulations of more complex programmable network behaviors. *iii) Protocol-independent design*: with custom headers to ensure seamless interoperability between P4 switches and end hosts. *iv) Seamless integration with ns-3*: with modular design, supporting *Waf* and *CMake* for streamlined development and deployment.

The paper is structured as follows. Section 2 reviews P4 development and related work on ns-3-based P4 implementations. Section 3 presents the design of P4sim, covering its switch model, simulation



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICNS3 2025, Osaka, Japan*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1517-4/25/08

<https://doi.org/10.1145/3747204.3747210>

¹<https://github.com/HapCommSys/p4sim>

²<https://github.com/HapCommSys/p4sim-artifact-icns3>

time representation, and custom header. Next, Section 4 discusses the implementation of key modules, followed by an in-depth performance and model evaluation in Section 5. Finally, Section 6 summarizes our findings and outlines directions for future work.

2 Background and Related Work

2.1 P4 Development

P4 is a domain-specific language for programming protocol independent packet forwarding on programmable network devices. It enables users to define custom headers with precise field structures and bit widths for parsing and modification. Several studies have explored P4-based programmable networks on various platforms, including hardware-based solutions such as Tofino switches [8], Field-Programmable Gate Array (FPGA)-based accelerators, and SmartNICs. Tofino, with fixed pipeline stages, is aimed at production environments, supporting throughput up to several Tbps. NetFPGA [9] offers greater flexibility for customizing hardware logic to support specialized network functions. Primarily optimized for specific hardware targets (e.g., DPDK), T4P4S [10] also offers high-speed, low-latency packet processing. However, it has limited tooling and debugging support, and a smaller user base. The most widely used software-based switch implementation is bmv2, the second-generation reference software switch developed by the P4 language consortium [11–13]. Primarily used for learning and debugging, bmv2 enables experimentation with P4 programs in emulated environments like Mininet [14]. However, Mininet and bmv2 have notable performance limitations [1, 2], which hinders the study of network performance in large-scale scenarios.

ns-3 is a powerful discrete-event network simulator designed enabling the modeling of complex and large-scale network topologies with high accuracy. With its extensive ecosystem of modules and support for both wired and wireless networks, ns-3 is widely used in large-scale research to evaluate and test new network protocols and systems in diverse, real-world scenarios. Supporting P4 in ns-3 presents a tremendous opportunity to enhance data plane programmability, offering fine-grained control over packet processing. This capability unlocks new potential for simulating and optimizing advanced network functions, making ns-3 an even more versatile tool for research in programmable networks. A desired P4 simulator in ns-3 needs to fulfill the following four technical requirements:

- R1 **Supporting queue management and packet scheduling** enables essential functionalities in simulated P4 switches, such as packet priority scheduling and management of the number of allocated buffers and buffer sizes.
- R2 **Enabling virtual-time representation** is critical to accurately performing data plane operations such as bufferbloat mitigation and congestion control. Virtual (or simulation) time³ is a simulation-specific concept advancing through discrete events [15]. Virtual time is fundamental for accurate network simulation by enabling precise modeling of transmission delays and scheduling [7].
- R3 **Supporting variety of modern P4 architectures** is required to align the simulator with real-world programmable

networking research, as P4 typically targets PSA for deployment in actual P4 switch processing pipelines.

- R4 **Supporting protocol-independent packet processing** facilitates the evaluation of P4 programs in diverse or forward-looking scenarios, where support for custom headers and flexible parsing logic is essential for innovation and experimentation.

2.2 Related Work

Existing efforts to model switch behavior in ns-3 can be broadly categorized into three types. First, switching behavior can be implemented through external modules, such as the OpenFlow switch module [16]. Although the module integrates with a controller library, its reliance on a bridge-based architecture and lack of native forwarding logic limit its adaptability for programmable networks. Second, dedicated switch models with custom scheduling and buffering mechanisms, such as iterative Serial Line Internet Protocol (iSLIP) and Combined Input and Crosspoint Buffered (CICB) architectures [17], provide more comprehensive simulation of data paths. Third, research efforts have modeled specific switch behaviors, such as congestion control [18] and in-band telemetry [19], often focusing on scenarios like Explicit Congestion Notification (ECN) or In-band Network Telemetry (INT) evaluation. Although these models improve switch simulation in targeted areas, they lack the flexibility to support fully programmable data planes like those enabled by P4.

There exists a few previous work to support P4 in ns-3. ns3-bmv2 [3] implements the P4 V1Model within ns-3's traffic control module for deployment on a single network path. It focuses on queueing and scheduling, but lacks core P4 functionalities like routing and switch-level processing, making it more suitable for traffic shaping than full-fledged P4 network simulations. NetDevices [4] leverages T4P4S, a retargetable P4 compiler, to transform P4 programs into C code for integration into ns-3 simulation scripts, enabling P4 switch modeling. However, this approach introduces complexity in integration, debugging, and traceability, with the generated code needing manual adjustments and lacking cohesive management within ns-3's modular framework, hindering maintenance and scalability. NS4 [5, 6], a P4-driven simulator, embeds the V1Model in ns-3 to support flow table configuration and programmable match-action processing in P4-enabled networks. However, it focuses mainly on packet forwarding, leveraging only a limited subset of bmv2 features and omitting time-aware functions like scheduling and delay modeling. Existing approaches integrate bmv2, which runs in real time, with ns-3's virtual time framework in a simplistic way, neglecting the effects of their mismatched time models. This compromises bmv2's native timing behavior, resulting in a lack of accurate timing representation. NS4 and ns-3 implementations omit a built-in switch queueing model, vital for a precise simulation of switch performance under load. NS4 and similar tools predominantly support bmv2's V1Model, excluding PSA and PNA architectures, which restricts their relevance for real-world programmable networking research where P4 commonly relies on PSA for switch pipeline deployment. In summary of this discussion, we recapitulate current software-based P4 implementations, as outlined in Table 1.

³we use *virtual time* and *simulation time* interchangeably

Table 1: Comparison of Software-Based P4 Platforms

Comparison Metrics	bmw2 [11]	p4c-dpdk [20] / t4p4s [10]	p4c-ebpf [21]	ns3-bmw2 [3]	NetDevices [4]	NS4 [5, 6]	P4sim (proposed)
Time Type	real time	real time	real time	virtual time	real time	virtual time	virtual time
Time Model [15]	✓	×	×	×	×	×	✓
P4 Functional Support	✓	✓	✓	◇	◇	◇	✓
P4 Version 16 and 14	✓	◇	◇	✓	✓	✓	✓
Performance	◇	✓	✓	✓	◇	✓	✓
Tracing & Logging	✓	◇	✓	✓	×	×	✓
P4 Switch Architecture	V1Model, PSA, PNA	PSA, PNA	PSA, eBPF backend	Incomplete V1Model	V1Model, PSA	Incomplete V1Model	V1Model, PSA, PNA

Note: ✓: Supported ◇: Partially Supported ×: Not Supported

In summary, none of the existing works fully meets all technical requirements (R1-R4), highlighting the need for a more comprehensive P4 simulator in ns-3. Developing such a simulator presents several key challenges. Supporting multiple P4 architectures requires re-architecting the switch pipeline to align with modern parsing, processing, and deparsing specifications. Furthermore, PSA introduces external components such as counters and meters, adding more complexity to both simulation and integration. Enabling custom headers presents another challenge, as existing net device and host models are tightly coupled with traditional protocol stacks (e.g., Ethernet II, IP, UDP/TCP), making it difficult to simulate custom protocols across the network. To address these challenges, we introduce P4sim, which allows simulating high-performance programmable switches, supports a variety of P4 architectures, and enables integrating new protocols and external components in a flexible manner.

3 P4sim Design

The P4sim design centers on a P4 switch that delivers programmable switch capabilities, complemented by port net devices. It features a protocol-independent header structure and utilizes full-duplex channels for underlying communication. We implement the P4 switch by incorporating external bmw2 functions into ns-3, adopting an approach similar to the OpenFlow switch module [16].

3.1 Modeling P4 Switch

The P4 switch in P4sim consists of two primary components. The hardware management module governs hardware-related switch functions and manages packet I/O with external nodes in ns-3. The programmable switching module executes internal packet processing as configured by user-defined P4 programs. This modular design, illustrated in Figure 1, decouples the P4-based processing of packets from the implementation of the underlying hardware, allowing independent development, testing, and extension of each component. This subsection examines the PSA architecture.

Figure 1 illustrates the internal structure of the proposed P4 switch model implemented in ns-3, following the PSA architecture. The figure highlights two main components: the programmable switching part (configured with P4) and the hardware management part (configured with ns-3). In that figure, the hardware management module is depicted as the gray box *Switch Configuration from*

ns-3 and the associated elements. Accurate switch modeling in network simulation requires accounting for forwarding rates, queue buffer sizes, scheduling, intra-device latency, and error models [22]. The P4 switch captures all these aspects except error modeling, consistent with prior work prioritizing switch behavior over link-layer errors. Details on forwarding rates, scheduling, and latency follow in the next subsection. The input buffer is a priority queue with two subqueues. External packets are assigned lower priority by default, whereas internal packets (e.g., recirculated or resubmitted) are prioritized to ensure rapid reprocessing. The queue buffer contains up to eight virtual queues per port, which are dequeued in strict priority order. The transmit buffer is omitted because it merely serves as a temporary staging area for outbound packets to the network, without adding any additional functionality. The output port could leverage ns-3’s *QueueDisc* from the traffic-control module, which efficiently supports queuing and scheduling mechanisms. This design, guided by Occam’s razor, avoids redundant components. The hardware buffer size is configurable via ns-3, with the actual usage adjustable through the control plane interface.

We now describe the programmable switching module, depicted as the *Switch Configuration from P4* box and associated components in Figure 1. The P4 switch includes the ingress and egress pipelines, each comprising a programmable parser, match-action unit, and deparser. The P4 program defines the behavior of the data plane for packet processing, while the control plane interface, using Thrift⁴, dynamically manages flow table entries by directly communicating with each P4 switch to insert or update them. Beyond basic packet buffering, the input and queue buffers support P4 primitive actions such as recirculate, resubmit, and clone. These actions, illustrated by gray arrows in the figure, indicate that packets are reinserted into the buffer for additional processing.

3.2 Simulation Time Representation

To enable virtual time representation in the P4 switch, we develop a mechanism integrating timestamping, scheduling, and timed packet release. We first calculate queueing delay from the switch’s maximum forwarding rate, then assess how virtual queue configurations influence this delay, and finally map timestamps to P4 control logic for time-sensitive packet processing. We address the absence of virtual time modeling in the P4 switch with an event-based periodic

⁴<https://thrift.apache.org>

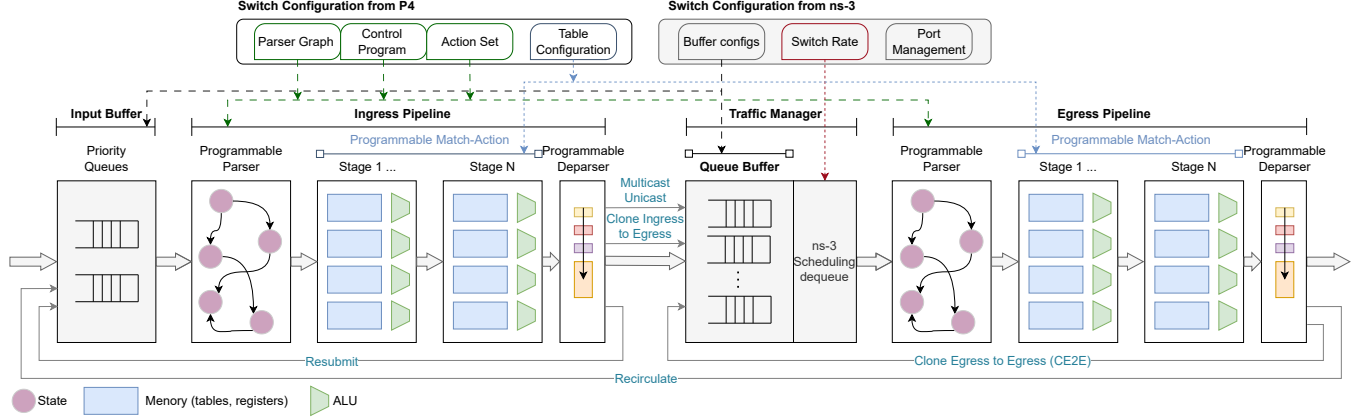


Figure 1: Internal Structure of the Proposed P4 Switch Model in ns-3

scheduling mechanism. This approach replicates internal timing by triggering events at intervals based on the switch’s maximum forwarding rate, as depicted in Figure 1. Each dequeue operation advances the simulation clock by a time step equal to the inverse of this rate. Although assigning separate schedulers to each virtual queue is feasible, this would generate multiple events per packet within a single switch, degrading efficiency. Instead, we centralize scheduling at the queue buffer level, ensuring each packet triggers only one event per switch. This method minimizes computational overhead and enhances processing performance. In contrast to traditional simulation time models that calculate delays from system states and trigger callbacks after fixed intervals, our model advances time through polling. It checks the eligibility of the packet at regular intervals, dequeuing the packets only when their expected dequeue time arrives. This polling-based approach simplifies implementation by bypassing complex mathematical modeling, though it introduces minor idle-time overhead.

We model how virtual queue configurations affect delay using ns-3’s simulation time. Each virtual queue employs a token-bucket mechanism to assign incoming packets an expected dequeue time, based on its own processing rate. This ensures that the dequeue rate for each virtual queue remains independent, different from the global scheduling rate of the switch. Users can modify queue rates and configurations during runtime via the control plane interface. Leveraging ns-3’s simulation time rather than real-time clocks, the model ensures precise and adaptable timing control. Queuing latency depends on key factors, including traffic arrival rate, queue rate (per queue and aggregate), queue length, and packet priority distribution.

The P4 processing component utilizes virtual timestamps from ns-3 to enable time-aware packet handling. An analysis of V1Model, PSA, and PNA highlights the necessity of intrinsic metadata, including ingress and egress timestamps, as well as queuing-related fields such as *enq_timestamp* (enqueue time) and *deq_timedelta* (queuing delay), to track packet processing stages. Additional queue-specific metadata, such as queue occupancy and priority, is also required for more accurate modeling.

Event-based scheduling operates within the queue buffer between the ingress and egress pipelines. Each packet’s enqueue and egress timestamps capture the queuing delay, which is accurately reflected in their difference, *deq_timedelta*. Moreover, both the ingress and egress timestamps are correctly assigned. This ensures precise time tracking within the P4 program’s scheduling logic. The time interval between the ingress and enqueue timestamps represents the ingress processing time, which includes match-action execution. In the absence of hardware testing data, we assume these timestamps are identical, but future improvements could integrate hardware-based delay models for more accurate simulations. By incorporating packet queuing and scheduling delays into the overall virtual-time representation, P4sim provides distinct timestamps at different processing stages within the P4 pipeline. These timestamps are then mapped to the P4 switch’s control logic to enable time-sensitive packet processing. As a result, P4sim satisfies requirement R2.

3.3 Supporting Modern P4 Architectures

In addition to PSA, modern P4 architectures also include V1Model and PNA. This section discusses the design considerations for supporting these three architectures in P4sim. V1Model provides a basic match-action pipeline and was initially introduced as the reference architecture for P4-16, the current version of the P4 programming language since 2016. PNA is tailored for high-performance Network Interface Card (NIC), enabling programmable packet processing for data center and in-network computing scenarios. bmv2 supports V1Model, PSA, and PNA, enabling simulation of various NIC and switch behaviors. These validated models can be reused in ns-3, simplifying development and ensuring semantic consistency of P4, making bmv2 a strong foundation for P4sim. Our design leverages the implementation of bmv2. As shown in Figure 1, the PSA architecture includes all stages of the packet processing pipeline. Similarly, the V1Model follows the overall pipeline structure but excludes the ingress deparser and egress parser components. The PNA architecture, originally intended for NIC modeling, is utilized in simulations akin to a multi-port switch to facilitate module reuse. The major distinctions from switch-based models lie in the structure of the P4 program and the packet processing logic. Consequently, we have

developed a net-to-net architecture featuring the main parser, main control, and main deparser components in the packet processing path. After processing, the packet is transmitted to the network for subsequent forwarding. By supporting V1Model, PSA, and PNA architectures, P4sim fulfills the requirement R3 for accommodating diverse modern P4 architectures.

3.4 Enabling P4 Custom Header

In ns-3, packet header processing follows a layered approach, with each protocol layer adding its respective header before transmission over the network. To streamline and standardize custom header definitions, we introduce an abstraction that categorizes headers based on their corresponding network layer. A custom header can be applied at the data link layer (*LAYER_2*), network layer (*LAYER_3*), or transport layer (*LAYER_4*). Users can control header placement by specifying one of three modes: *ADD_BEFORE* (insert before an existing header), *ADD_AFTER* (insert after an existing header), or *REPLACE* (substitute an existing header). This approach enhances flexibility in modifying packet structures, facilitating protocol experimentation, and enabling the rapid prototyping of novel networking concepts in ns-3 simulations.

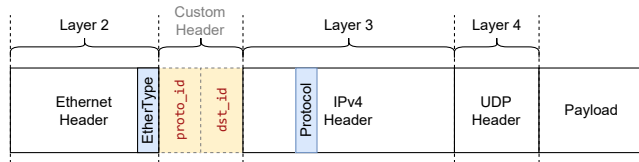


Figure 2: Packet With Tunnel Custom Header

To illustrate this approach, Figure 2 shows how a custom tunnel header is inserted between the Ethernet and IPv4 headers. This placement is defined using *LAYER_3* and *ADD_BEFORE*. Listing 1 provides a code implementation of this approach, where the tunnel header includes fields for the protocol ID (*proto_id*) and destination ID (*dst_id*), each 16 bits long. To ensure correct parsing and debugging, the custom header must include the *proto_id* field to indicate the type of the next header. Without it, the receiver's parser may misinterpret the protocol, causing a parsing error. Custom header processing is implemented at the netdevice level, enabling dynamic modification of headers of layers 2 to 4 without affecting the upper layer protocols. The sender assembles headers sequentially, while the receiver parses them based on protocol fields. All hosts must use the custom netdevice for correct header processing.

```

1 CustomHeader myTunnelHeader;
2 myTunnelHeader.SetLayer (LAYER_3);
3 myTunnelHeader.SetOperator (ADD_BEFORE);
4 myTunnelHeader.AddField ("proto_id", 16); // 16 bits
5 myTunnelHeader.AddField ("dst_id", 16); // 16 bits
6 myTunnelHeader.SetField ("proto_id", 0x0800); // IPv4
7 myTunnelHeader.SetField ("dst_id", 0x22);

```

Listing 1: Tunnel Custom Header Definition

3.5 P4 Switch and Channel

Currently, ns-3 and NS4 lack a dedicated Ethernet switch module. The closest alternative is the *ns3::BridgeNetDevice*, which functions

as a virtual device that connects multiple LAN segments. However, it behaves more like a hub than a true Ethernet switch because it forwards packets to all connected *ns3::NetDevice* without performing forwarding decisions based on packet headers. To enable packet processing and forwarding, the P4 switch is built on top of the bridge module and extends it with a modified channel and programmable logic with P4. In ns-3, only Carrier Sense Multiple Access (CSMA) and Point-to-Point (P2P) links simulate high-speed Ethernet, but neither is compatible with a P4 switch. The CSMA link operates in half-duplex mode. When using this channel, the switch behaves as a hub, broadcasting packets to all connected nodes without making any forwarding decisions. On the other hand, the P2P link represents a full-duplex connection, typically modeled after RS-232 or RS-422 and using the Point-to-Point Protocol (PPP). To address this gap, we design a dedicated P2P channel model, replacing the PPP with Ethernet II. Also implement the processing for the custom header. By replacing the CSMA channel with a modified P2P channel, the switching system have the duplex channel. Address Resolution Protocol (ARP) is disabled by default. Developers can enable ARP by themselves, but they also need to deploy ARP processing functions in the P4 program. The P4 switch can be flexibly configured via P4 programs to operate as either a Layer-3 router or a Layer-2 switch.

4 P4sim Implementation

In this section describes the implementation of P4 switch in ns-3, with a particular emphasis on the key classes involved in packet processing. We show new classes aligned with the Unified Modeling Language (UML) diagram presented in Figure 3. It concisely illustrates the essential relationships, dependencies, and attributes of the core classes of P4sim.

4.1 P4SwitchNetDevice

The *P4SwitchNetDevice* class serves as the underlying switch hardware management component in the P4 switch. It extends the *ns3::NetDevice* base class and is designed to support multiple network ports and hardware configurations. This switch device can potentially incorporate *CustomP2PNetDevice* in Figure 3 as a port, where the corresponding channel should be the *P4P2PChannel*. Additionally, it can be configured with *ns3::CsmaNetDevice* as a port, the channel should be set to a CSMA channel. It supports dynamic port management, allowing developers to add or remove ports as needed without restarting the P4 switch or reconfiguring the switch from P4. The primary responsibilities of *P4SwitchNetDevice* include packet reception, transmission, and initializing the *P4SwitchCore* for P4-based processing. In the case of the V1Model architecture, each *P4SwitchNetDevice* is associated with a dedicated *P4CoreV1model* processing module. To simplify deployment, the *P4Helper* utility automates the installation and configuration of *P4SwitchNetDevice*, enabling seamless integration within network simulation environments.

4.2 P4SwitchCore

The *P4SwitchCore* class serves as the programmable switching part of the switch, managing P4-specific implementations such as packet processing and forwarding logic. It handles key functionalities,

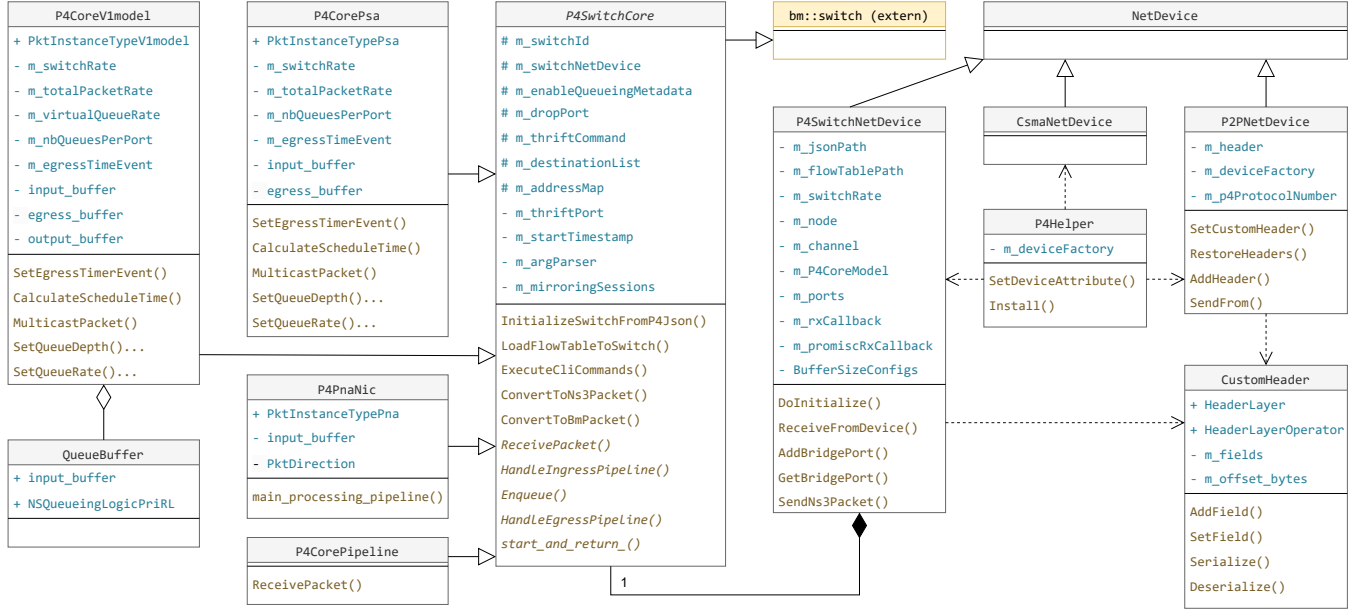


Figure 3: UML Diagram of P4 Switch Implementation

including queue management and scheduling. *P4SwitchCore* module extends the external *bm::switch* and implements its functionality within the ns-3 network simulation framework. The *P4SwitchCore* processes external packets received from the *P4SwitchNetDevice* and forwards them to the appropriate output port. It has several subclass implementations, each corresponding to a specific architecture, such as V1Model from *P4CoreV1model*, PSA from *P4CorePsa*, and PNA from *P4PnaNic*. It is initialized using a compiled P4 JSON file, which contains configuration and mapping information for various components defined in the P4 language. This includes the parser, tables, actions for ingress and egress, the deparser, and configurations for registers and other resources. After initialization, the system enables the Thrift port, allowing flow table entries to be configured dynamically via the Thrift interface. The simulation of ns-3 waits for the entries to be configured before proceeding.

4.3 CustomHeader and CustomP2PNetDevice

The *CustomHeader* class implements user-defined headers, allowing users to define a header and bind it to *CustomP2PNetDevice* (*CPDevice*), which then provides support for the custom header. Inheriting from the *Header* class, *CustomHeader* supports arbitrary custom fields and can be placed at layers 2-4. Additionally, it allows setting specific values for corresponding fields within the data flow header. To support this header on both the sender and receiver sides, we introduce functions for custom header insertion and extraction in *CPDevice*. The original PPP on the default net device has been replaced with the Ethernet II protocol. In particular, the Frame Check Sequence (FCS) is omitted from the transmitted packets. This implies that the *CPDevice* functions as an Ethernet port. We define the serialization of the header, and during deserialization, we traverse and extract each field based on its *bitWidth*. Additionally, we introduce the *P4P2PChannel* as its high-speed full-duplex connection. To

enable *CPDevice* to add or retain the original application-generated traffic automatically, we design a mechanism that dynamically determines header insertion based on destination port ranges. Traffic with UDP or TCP destination port numbers between 10000 and 12000 is augmented with a custom header, while traffic outside this range is transmitted without modification. The port numbers for TCP or UDP traffic can be easily configured in ns-3 using the application helper. This automation reduces the need for manual configuration, making it easier for researchers to validate P4-based header modifications in ns-3 simulations.

The P4sim implementation is compatible with both the *Waf* build system (used in ns-3.35 and earlier) and the *CMake* build system (introduced in ns-3.39), and is compiled and tested with C++17.

5 Evaluation

This section evaluates the adherence of P4sim to the technical requirements R1-R4. We measure P4sim's throughput and simulation execution time to benchmark its efficiency. To demonstrate the practical use of P4sim, we implement two common networking scenarios: load balancing and tunneling.

5.1 Performance and Runtime Overhead

We evaluate P4sim's performance using the fundamental L3 program *ipv4_forward*, implemented on the V1Model architecture. This enables a precise and focused assessment of the core performance characteristics in a controlled environment. To isolate forwarding performance, we use a simple topology in which the P4 switch forwards packets between two hosts based on IPv4 addresses. We evaluate the P4 switch model using CSMA channels, which are supported in wired networks via the ns-3 bridge. We compare the performance of three types of switches: our P4sim P4 switch, the

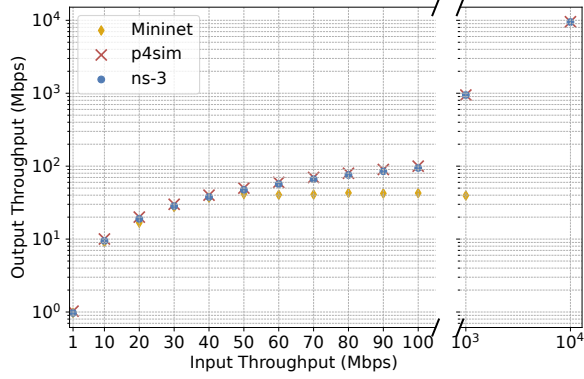


Figure 4: Evaluation of P4 Switch Performance in Packet Forwarding

bmw2 software switch in Mininet, and the standard ns-3 bridge (without P4 support). Performance and efficiency are measured in terms of throughput and simulation execution (or wall-clock) time. To ensure consistency, we repeat each experiment 10 times, measuring functional correctness and feasibility.

Figure 4 shows the average output throughput as input traffic loads increase from 1 Mbps to 10 Gbps. Mininet’s output throughput saturates at approximately 43 Mbps, creating a performance bottleneck where increasing the input traffic rate no longer improves throughput. In contrast, P4sim and ns-3 maintain near-linear throughput scaling, accurately reflecting the input traffic rate across all configurations. These results demonstrate that P4sim reliably simulates high-speed networks by accurately preserving throughput across various traffic rates, fulfilling requirement R1 for performance evaluation.

To complement the performance analysis, we evaluate simulation efficiency, a key factor for modeling large-scale programmable networks. Specifically, we assess the runtime overhead of programmable data plane functions in ns-3. Efficiency is assessed on the basis of the wall-clock time required to complete each simulation run. Figure 5 compares the wall-clock time of ns-3 and P4sim for a fixed 1-second simulation run, using the same input throughput to ensure a fair comparison. Despite differences in absolute execution time, P4sim and ns-3 exhibit similar growth trends in simulation time across all traffic values, as shown in the logarithmic plot. This indicates a comparable growth rate in wall-clock time as traffic increases. The consistent vertical offset of approximately 5 dB indicates that P4sim runs approximately 3.16 times slower than ns-3. This stable ratio confirms that while both simulators scale similarly with increasing traffic, P4sim consistently requires approximately three times the wall-clock execution time of ns-3. The discrepancy arises from P4sim’s initialization overhead, which includes startup procedures and flow table deployment. These operations occur once per simulation, with the execution time scaling primarily with the number of P4 switch instances. The simulation execution time increases linearly with the input traffic, which is expected given that more packets require additional processing and event handling during the simulation. This scenario demonstrates

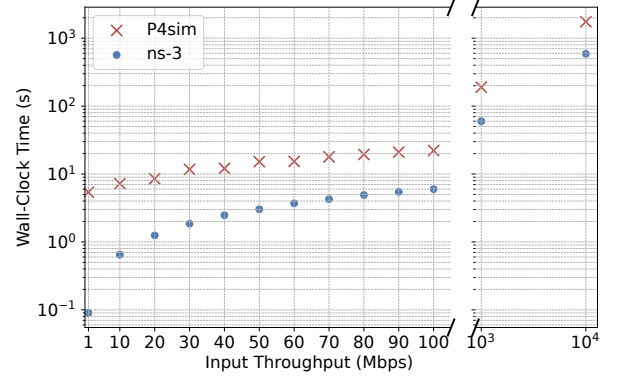


Figure 5: Simulation Efficiency Based on Execution Time

that the programmable data plane in P4sim does not significantly increase runtime overhead, thus satisfying requirement R1.

5.2 Load Balancing

Load balancing is crucial in modern networks to minimize congestion and improve performance. In data centers, spine-leaf architectures provide scalability, high availability, and redundant paths for East-West traffic. Leaf switches connect to servers, while spine switches interconnect leaf switches for efficient routing. To evaluate our P4-based load balancing design, we implemented a use case (Figure 6) using the *CustomP2PNetDevice* and *P4P2PChannel* in ns-3 to simulate P4 switches in a spine-leaf architecture.

In the setup (Figure 6), host h1 generates 1000 UDP flows, each targeting a different destination port, with each flow operating at 10 Mbps, resulting in a total transmission rate of 10 Gbps. Traffic follows an On-Off pattern using the *OnOffApplication*, with exponentially distributed ON and OFF periods having mean values of 2 and 1 second, respectively. The traffic is transmitted via the *P4P2PChannel* to the destination host, h4. To ensure the network can accommodate the full traffic load, we configure the link using *CustomP2PNetDevice* with a data rate of 10 Gbps and set the channel delay to 10 ns. For load balancing, all switches use static hash-based ECMP routing, applying a 5-tuple hash function to incoming packets to distribute traffic evenly across available paths. We measure

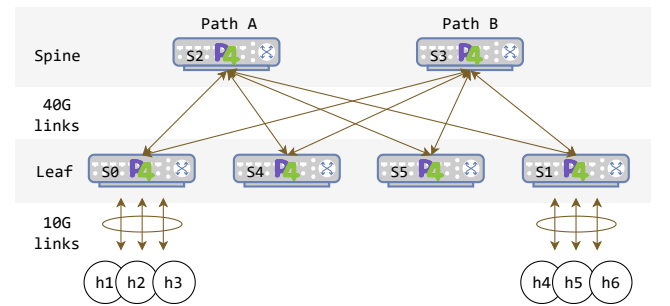


Figure 6: Spine-Leaf Network Topology for ECMP Load Balancing [23]

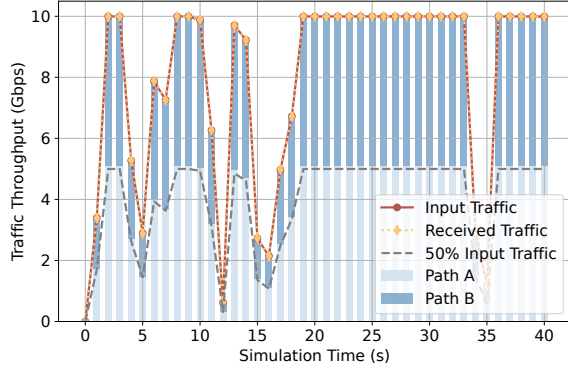


Figure 7: Throughput of Hosts and Paths in Load-Balancing System

traffic throughput at key nodes, with input and received traffic recorded at the leaf nodes S0 and S1, and traffic on two paths (A and B) measured through spine nodes S2 and S3.

The load balancing simulation results are shown in Figure 7. The red line represents the input traffic from host h1, while the yellow dashed line shows the received throughput at host h4. The light and dark blue bars represent throughput measured at switches S2 and S3 on Paths A and B, respectively. A gray dashed line at 50% of the input traffic serves as a reference for balanced distribution between the two paths. From 0 to 40 seconds, input and received traffic fluctuate between 0 and 10 Gbps, with the combined throughput of Path A and Path B matching the input traffic. While the load is nearly balanced, Path B consistently carries slightly more traffic than Path A, with an average throughput ratio of 1.0458 over the 40-second period. These results show that the P4-based load balancing in P4sim functions effectively, with the total received throughput matching input traffic. The near-equal distribution of traffic across both paths confirms the efficacy of the hash-based ECMP strategy. The slight imbalance is likely due to the limited number of flows (1000), which is insufficient for perfect distribution using the 5-tuple hash. Overall, the experiment confirms that P4sim supports reliable programmable data plane performance (R1) in a realistic load balancing scenario.

5.3 Tunneling

Tunneling is commonly used in network virtualization, overlay networks, and service function chaining, enabling packet encapsulation for forwarding across intermediate nodes. This technique is key to building flexible, multi-layered network architectures. To assess P4sim’s support for protocol-independent processing, we implement a tunneling use case with custom header parsing and encapsulation-based forwarding. We extend a basic IP router by adding a custom tunneling mechanism. The custom header, *myTunnel_t*, includes a protocol identifier and destination ID to guide forwarding decisions. This use case highlights P4sim’s ability to process user-defined headers and dynamic forwarding logic.

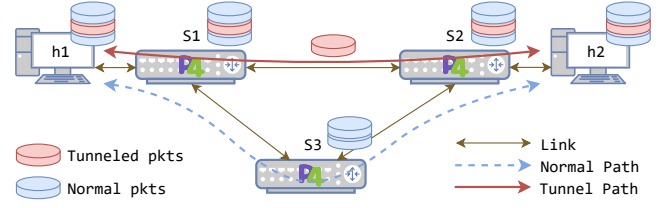


Figure 8: Data Transmission in P4 Network: Normal vs. Tunnel Traffic [24]

The *myTunnel_t* header consists of two fields: a protocol identifier (*proto_id*) for the payload type (e.g., IPv4 header) and a destination ID (*dst_id*) for lookup-based forwarding. Its structure is shown in Listing 1. The P4 switch is updated to parse this header, extract its fields based on the Ethernet protocol type (assumed to be 0x1212), and apply forwarding actions accordingly. A new *myTunnel_exact* table performs exact matching on the *dst_id* field to trigger forwarding. The ingress pipeline prioritizes tunnel-based forwarding for packets with the *myTunnel_t* header and defaults to standard IPv4 forwarding for others. This hybrid approach enables seamless support for both encapsulated and non-encapsulated flows. The deparser then reconstructs the packet in the correct order: Ethernet, *myTunnel_t*, and inner IP header, ensuring compatibility with downstream devices.

In this scenario, h1 transmits tunneled traffic at 10 Mbps with 1250 packets per seconds (*pps*) and normal traffic at 40 Mbps with 5000 *pps* to h2. Normal traffic follows the path S1, S3, S2, h2, while tunneled traffic takes a direct route through S1, S2, h2. Packet characteristics are captured in *pcap* files, though custom headers may not be parsed automatically in Wireshark, affecting throughput calculations in *bps*. To avoid inaccuracies, we use *pps* as the evaluation metric. The benchmark results show that S1 forwards approximately 5000 *pps* to S3 and 1250 *pps* to S2, with S2 receiving a total average of 6250 *pps*. This confirms that the tunnel header operates as expected, ensuring the configured traffic is transmitted and forwarded at the correct rate. The successful establishment of the tunnel in the network demonstrates the correct implementation of the custom header, thereby fulfilling the requirements of R4.

5.4 Discussion

In addition to the use cases discussed above, we contribute several more examples, including *firewall*, *qos* (Quality of Service), and *fattree*. In the *firewall* example, a stateful firewall is implemented using a Bloom filter and registers to block unauthorized flows based on historical state, verifying correct register usage in P4. The *qos* example maps flows to different priority queues under congestion, demonstrating end-to-end delay differentiation and compliance with requirement R2. The *fattree* example allows users to configure the topology size via a parameter *K* (e.g., 2, 4, 6), automatically generating the network and corresponding forwarding rules including ARP and IP. With *K* = 6, the system simulates 45 P4 switches and completes a 3-second run in about one minute. Additional details and configurations are available on our artifact gitHub repo. Finally, we include three architecture-specific examples *simple_psa*,

simple_v1model, and *simple_pna* to ensure deployment correctness across different P4 targets, fulfilling requirement R3.

Limitations. Despite advancements in the P4 switch model, it remains unaware of link conditions such as channel state and port queue occupancy, which limits its ability to adapt to dynamically changing network conditions. Currently, congestion is simulated by statically configuring dequeue rates based on predefined link conditions, which excludes the dynamic nature of congestion feedback in real networks.

6 Conclusion and Future Work

This paper introduced P4sim, an enhanced P4-driven simulation framework that addresses the limitations of existing platforms in high-speed, large-scale network simulations. Built on NS4, it aligns with ns-3's modular design while improving queue modeling, time scheduling, and expanding support for V1Model, PSA, and PNA architectures. Additionally, protocol-independent custom headers enable seamless interaction between P4-enabled hosts and switches, enhancing flexibility and ease of integration. Our evaluation demonstrated the efficiency and precision of P4sim in packet processing and queue management, validating its practicality through tunneling and load-balancing use cases. The results showed that P4sim improves simulation fidelity and scalability compared to existing solutions, making it a valuable tool to advance research and education on programmable networking.

Future work will include integrating queue state awareness into the switch model. This allows the P4 switch to adapt to dynamically changing network conditions. Furthermore, we plan to develop models of real-world P4 hardware devices, such as smartNIC. Enhancing interoperability with emerging programmable networking frameworks would further extend its applicability. By addressing these challenges, P4sim can serve as a foundation for more accurate, scalable, and intelligent network simulations, helping researchers and practitioners push the boundaries of programmable data planes.

Acknowledgments

Funded by the German Research Foundation (DFG, Deutsche Forschungsgemeinschaft) as part of Germany's Excellence Strategy – EXC 2050/1 – Project ID 390696704 – Cluster of Excellence “Centre for Tactile Internet with Human-in-the-Loop” (CeTI) of Technische Universität Dresden.

References

- [1] P4 Community. 2019. *Performance of bmv2*. P4 Language Consortium. <https://github.com/p4lang/behavioral-model/blob/main/docs/performance.md>
- [2] Kazumi Kumazoe, Masahito Shibata, and Masato Tsuru. 2024. Experimental Comparison of Software Switches for Data Plane Programming Language P4-BMv2 and T4P4S-. In *Proceedings of the 2023 12th International Conference on Networks, Communication and Computing* (Osaka, Japan) (ICNCC '23). Association for Computing Machinery, New York, NY, USA, 210–217. doi:10.1145/3638837.3638871
- [3] PIFO-TM Github organization. n.d. *ns3-bmv2*. PIFO-TM. <https://github.com/PIFO-TM/ns3-bmv2>
- [4] mrosan. 2017. *mrosan/P4SwitchNetDevice*. mrosan. <https://github.com/mrosan/P4SwitchNetDevice>
- [5] Chengze Fan, Jun Bi, Yu Zhou, Cheng Zhang, and Haisu Yu. 2017. NS4: A P4-driven Network Simulator. In *Proceedings of the SIGCOMM Posters and Demos* (Los Angeles, CA, USA) (SIGCOMM Posters and Demos '17). Association for Computing Machinery, New York, NY, USA, 105–107. doi:10.1145/3123878.3132002
- [6] Jiasong Bai, Jun Bi, Peng Kuang, Chengze Fan, Yu Zhou, and Cheng Zhang. 2018. NS4: Enabling Programmable Data Plane Simulation. In *Proceedings of the Symposium on SDN Research* (Los Angeles, CA, USA) (SOSR '18). Association for Computing Machinery, New York, NY, USA, Article 12, 7 pages. doi:10.1145/3185467.3185470
- [7] David R. Jefferson. 1985. Virtual Time. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 7, 3 (1985), 404–425.
- [8] Barefoot Networks. 2016. *Tofino ASIC*. Intel Corporation. <https://www.intel.com/content/www/us/en/products/details/network-io/intelligent-fabric-processors/tofino.html>
- [9] Noa Zilberman, Yury Audzevich, G. Adam Covington, and Andrew W. Moore. 2014. NetFPGA SUME: Toward 100 Gbps as Research Commodity. *IEEE micro* 34, 5 (2014), 32–41.
- [10] Péter Vörös, Dániel Horpácsi, Róbert Kitlei, Dániel Leskó, Máté Tejfel, and Sándor Laki. 2018. T4P4S: A Target-Independent Compiler for Protocol-Independent Packet Processors. In *2018 IEEE 19th International Conference on High Performance Switching and Routing (HPSR)*. IEEE, Bucharest, Romania, 1–8. doi:10.1109/HPSR.2018.8850752
- [11] P4 Community. n.d. *p4lang/behavioral-model*. P4 Language Consortium. <https://github.com/p4lang/behavioral-model>
- [12] Athanasios Liatifis, Panagiotis Sarigiannidis, Vasileios Argyriou, and Thomas Lagkas. 2023. Advancing SDN from Openflow to P4: A Survey. *Comput. Surveys* 55, 9 (2023), 1–37.
- [13] Frederik Hauser, Marco Häberle, Daniel Merling, Steffen Lindner, Vladimir Gurevich, Florian Zeiger, Reinhard Frank, and Michael Menth. 2023. A Survey on Data Plane Programming with P4: Fundamentals, Advances, and Applied Research. *Journal of Network and Computer Applications* 212 (2023), 103561.
- [14] P4 Community. n.d. *p4lang/Tutorials with Mininet and bmv2*. P4 Language Consortium. <https://github.com/p4lang/tutorials>
- [15] Tomáš Martinek, Mauro Campanella, Federico Pederzoli, and Joseph Hill. 2022. *White Paper: Timestamping and Clock Synchronisation in P4-Programmable Platforms*. White Paper. GÉANT, Amsterdam, The Netherlands. <https://resources.geant.org>
- [16] Luciano Jerez Chaves, Islene Calciolari Garcia, and Edmundo Roberto Mauro Madeira. 2016. OFSwitch13: Enhancing ns-3 with OpenFlow 1.3 Support. In *Proceedings of the 2016 Workshop on ns-3* (Seattle, WA, USA) (WNS3 '16). Association for Computing Machinery, New York, NY, USA, 33–40. doi:10.1145/2915371.2915381
- [17] Xia Yu, Zeng Huaxin, and Shen Zhijun. 2009. Design and Implementation of Switch Module for ns-3. In *Proceedings of the Fourth International ICST Conference on Performance Evaluation Methodologies and Tools* (Pisa, Italy) (VALUETOOLS '09). ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering), Brussels, BEL, Article 3, 10 pages. doi:10.4108/ICST.VALUETOOLS2009.7658
- [18] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. 2019. HPCC: High Precision Congestion Control. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (SIGCOMM '19). Association for Computing Machinery, New York, NY, USA, 44–58. doi:10.1145/3341302.3342085
- [19] Ran Ben Basat, Sivaramakrishnan Ramanathan, Yuliang Li, Gianni Antichi, Minian Yu, and Michael Mitzenmacher. 2020. PINT: Probabilistic In-band Network Telemetry. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication* (Virtual Event, USA) (SIGCOMM '20). Association for Computing Machinery, New York, NY, USA, 662–680. doi:10.1145/3387514.3405894
- [20] P4 Community. n.d. *P4 DPDK Target Components*. P4 Language Consortium. <https://github.com/p4lang/p4-dpdk-target>
- [21] P4 Community. n.d. *P4 Ebp4 Backends*. P4 Language Consortium. <https://github.com/p4lang/p4c/tree/main/backends/ebp4>
- [22] Roman Chertov, Sonia Fahmy, and Ness B. Shroff. 2008. A Device-Independent Router Model. In *IEEE INFOCOM 2008 - The 27th Conference on Computer Communications*. IEEE, Phoenix, AZ, USA, 1642–1650. doi:10.1109/INFOCOM.2008.225
- [23] P4 Community. 2018. *P4 Language Tutorials Load Balancing Exercises*. P4 Language Consortium. https://github.com/p4lang/tutorials/tree/master/exercises/load_balance
- [24] P4 Community. 2018. *P4 Language Tutorials Basic Tunnel Exercises*. P4 Language Consortium. https://github.com/p4lang/tutorials/tree/master/exercises/basic_tunnel